

VQC Hyperparameter-Analyse - Breast Cancer

Modell: VQC (RealAmplitudes)

Datensatz: Breast Cancer (2 Klassen, 2 Features nach PCA) - identische Pipeline wie `07_vergleich_breast_cancer_fair.ipynb`

Analyse: 3 Optimizer $\times$ 4 Iterationsstufen = 12 Läufe

Ausgabe: `ergebnisse_hyperparam_bc.csv` (separat)

Vergleichbar mit `06_vqc_hyperparam_analyse.ipynb` (Iris) - binäre Klassifikation vs. 3-Klassen.

0. Imports

```
In [2]: import sys, os
sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, f1_score

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit.primitives import StatevectorSampler
from qiskit_machine_learning.optimizers import COBYLA, SPSA, ADAM

print("Imports OK")
```

Imports OK

1. Datensatz - identische Pipeline

Exakt gleiche Konfiguration wie `04_breast_cancer_fair.ipynb`.

```
In [2]: bc = load_breast_cancer()
X = bc.data
y = bc.target

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.3, random_state=42, stratify=y
)

pca = PCA(n_components=2, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train_pca)
X_test_sc = scaler.transform(X_test_pca)

print(f"Erklärte Varianz: {pca.explained_variance_ratio_.sum():.2%}")
print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")

```

Erklärte Varianz: 99.93%
Train: (398, 2) | Test: (171, 2)

2. Analyse-Loop

12 Läufe: 3 Optimizer × 4 Iterationsstufen (50, 100, 150, 200).
Ergebnisse werden laufend gespeichert.

```

In [3]: CSV_PATH = "Ergebnisse/ergebnisse_hyperparam_bc.csv"

OPTIMIZERS = {
    "COBYLA": lambda n: COBYLA(maxiter=n),
    "SPSA": lambda n: SPSA(maxiter=n),
    "ADAM": lambda n: ADAM(maxiter=n),
}

ITERATIONEN = [50, 100, 150, 200]

total = len(OPTIMIZERS) * len(ITERATIONEN)
run = 0

for opt_name, opt_fn in OPTIMIZERS.items():
    for iters in ITERATIONEN:
        run += 1
        print(f"[{run}/{total}] {opt_name} - {iters} Iterationen ...", end="

        feature_map = zz_feature_map(feature_dimension=2, reps=1)
        ansatz = real_amplitudes(num_qubits=2, reps=2)

        vqc = VQC(
            feature_map=feature_map,
            ansatz=ansatz,
            optimizer=opt_fn(iters),
            sampler=StatevectorSampler(),
        )

        start = time.time()
        vqc.fit(X_train_sc, y_train)
        train_time = round(time.time() - start, 4)

```

```

start = time.time()
y_pred = vqc.predict(X_test_sc)
infer_time = round(time.time() - start, 4)

acc = round(accuracy_score(y_test, y_pred), 4)
f1 = round(f1_score(y_test, y_pred, average="binary"), 4)

print(f"Accuracy: {acc:.4f} F1: {f1:.4f} ({train_time}s)")

entry = {
    "Modell": "VQC (RealAmplitudes)",
    "Datensatz": "Breast Cancer (2 Klassen, 2 Features, fair)",
    "Backend": "AerSimulator",
    "Optimizer": opt_name,
    "Iterationen": iters,
    "Accuracy": acc,
    "F1": f1,
    "Trainingszeit_s": train_time,
    "Inferenzzeit_s": infer_time,
}

df_entry = pd.DataFrame([entry])
if os.path.exists(CSV_PATH):
    df_entry.to_csv(CSV_PATH, mode="a", header=False, index=False)
else:
    df_entry.to_csv(CSV_PATH, index=False)

print()
print("Alle Läufe abgeschlossen.")

```

[1/12] COBYLA - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8889 F1: 0.9163 (141.0248s)

[2/12] COBYLA - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8655 F1: 0.8987 (203.7445s)

[3/12] COBYLA - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8596 F1: 0.8957 (205.7143s)

[4/12] COBYLA - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8480 F1: 0.8860 (165.4734s)

[5/12] SPSA - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8830 F1: 0.9123 (421.1541s)

[6/12] SPSA - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8830 F1: 0.9123 (698.5822s)

[7/12] SPSA - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8655 F1: 0.9004 (977.191s)
[8/12] SPSA - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.8596 F1: 0.8947 (1253.9019s)
[9/12] ADAM - 50 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.3509 F1: 0.3729 (2299.2241s)
[10/12] ADAM - 100 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.2105 F1: 0.1615 (4635.5254s)
[11/12] ADAM - 150 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.5029 F1: 0.5550 (7031.4684s)
[12/12] ADAM - 200 Iterationen ...

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Accuracy: 0.1930 F1: 0.1375 (9228.0303s)

Alle Läufe abgeschlossen.

3. Ergebnisübersicht

```
In [4]: df = pd.read_csv(CSV_PATH)
print(df[["Optimizer", "Iterationen", "Accuracy", "F1", "Trainingszeit_s"]].
```

Optimizer	Iterationen	Accuracy	F1	Trainingszeit_s
COBYLA	50	0.8889	0.9163	141.0248
COBYLA	100	0.8655	0.8987	203.7445
COBYLA	150	0.8596	0.8957	205.7143
COBYLA	200	0.8480	0.8860	165.4734
SPSA	50	0.8830	0.9123	421.1541
SPSA	100	0.8830	0.9123	698.5822
SPSA	150	0.8655	0.9004	977.1910
SPSA	200	0.8596	0.8947	1253.9019
ADAM	50	0.3509	0.3729	2299.2241
ADAM	100	0.2105	0.1615	4635.5254
ADAM	150	0.5029	0.5550	7031.4684
ADAM	200	0.1930	0.1375	9228.0303

4. Pivot-Tabellen

```
In [5]: pivot_acc = df.pivot(index="Optimizer", columns="Iterationen", values="Accuracy")
print("Accuracy:")
print(pivot_acc.to_string())
print()
pivot_f1 = df.pivot(index="Optimizer", columns="Iterationen", values="F1")
```

```

print("F1:")
print(pivot_f1.to_string())
print()
pivot_time = df.pivot(index="Optimizer", columns="Iterationen", values="Trainingszeit (s)")
print("Trainingszeit (s):")
print(pivot_time.to_string())

```

Accuracy:

Iterationen	50	100	150	200
Optimizer				
ADAM	0.3509	0.2105	0.5029	0.1930
COBYLA	0.8889	0.8655	0.8596	0.8480
SPSA	0.8830	0.8830	0.8655	0.8596

F1:

Iterationen	50	100	150	200
Optimizer				
ADAM	0.3729	0.1615	0.5550	0.1375
COBYLA	0.9163	0.8987	0.8957	0.8860
SPSA	0.9123	0.9123	0.9004	0.8947

Trainingszeit (s):

Iterationen	50	100	150	200
Optimizer				
ADAM	2299.2241	4635.5254	7031.4684	9228.0303
COBYLA	141.0248	203.7445	205.7143	165.4734
SPSA	421.1541	698.5822	977.1910	1253.9019

5. Vergleich mit Iris-Hyperparameter-Analyse

Direkter Gegenüberstellung: Binäre Klassifikation vs. 3-Klassen.

In [7]: CSV_IRIS = "Ergebnisse/ergebnisse_hyperparam.csv"

```

if os.path.exists(CSV_IRIS):
    df_iris = pd.read_csv(CSV_IRIS)
    df_bc = pd.read_csv(CSV_PATH)

    # Beste Accuracy je Optimizer
    best_iris = df_iris.groupby("Optimizer")["Accuracy"].max().rename("Iris")
    best_bc = df_bc.groupby("Optimizer")["Accuracy"].max().rename("Breast")

    print("Beste Accuracy je Optimizer:")
    print(pd.concat([best_iris, best_bc], axis=1).to_string())
else:
    print("ergebnisse_hyperparam.csv nicht gefunden - erst 06 ausführen.")

```

Beste Accuracy je Optimizer:

	Iris (3 Klassen)	Breast Cancer (2 Klassen)
Optimizer		
ADAM	0.3778	0.5029
COBYLA	0.5556	0.8889
SPSA	0.5556	0.8830